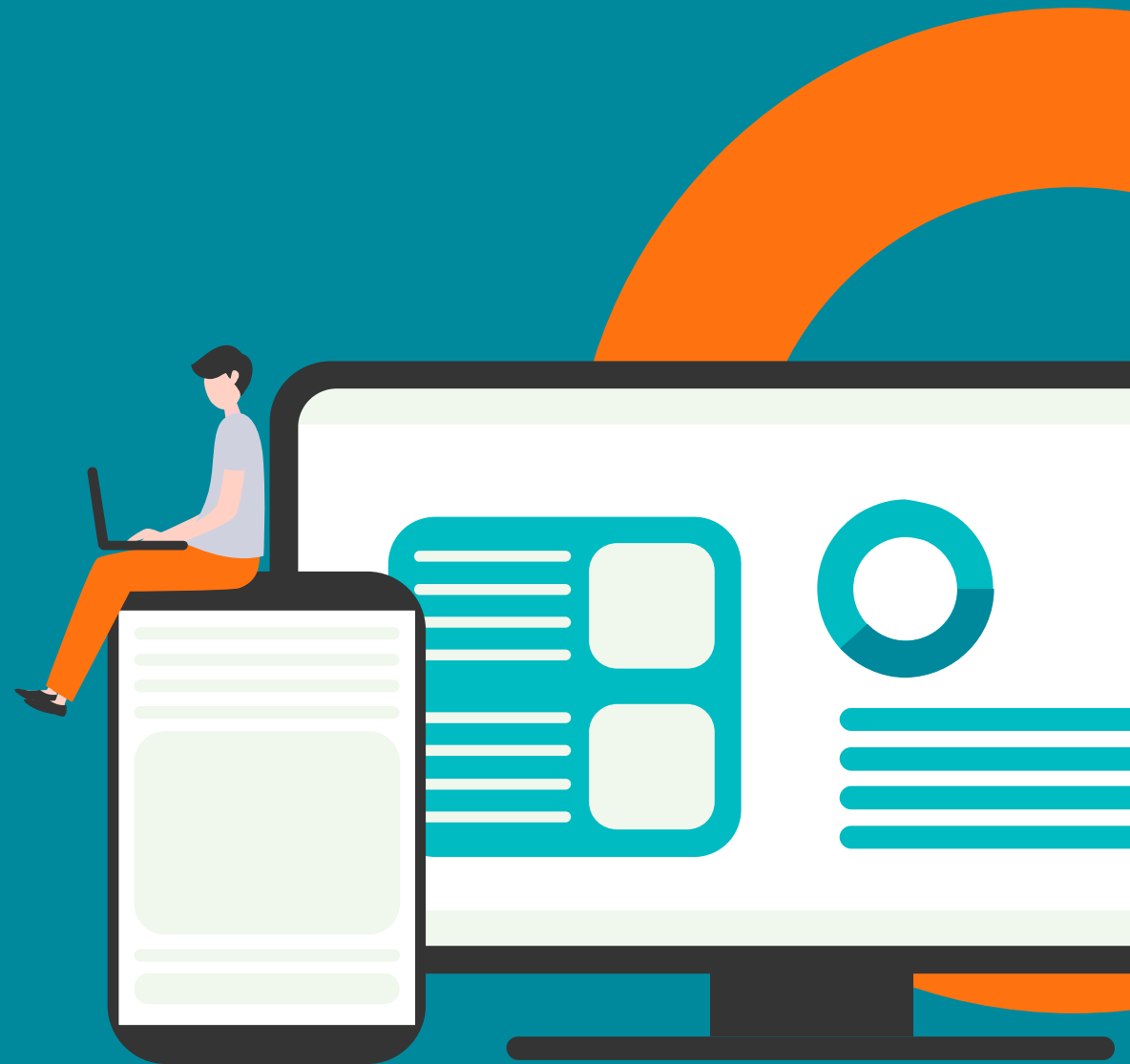




Proiectarea sistemelor informaticice

Seminar 4 - Limbajul UML
Diagrama claselor



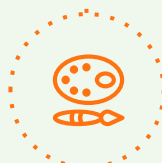
Definirea unei clase



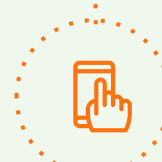
Ansamblu de obiecte care au aceleași **caracteristici** și **contrângeri**.



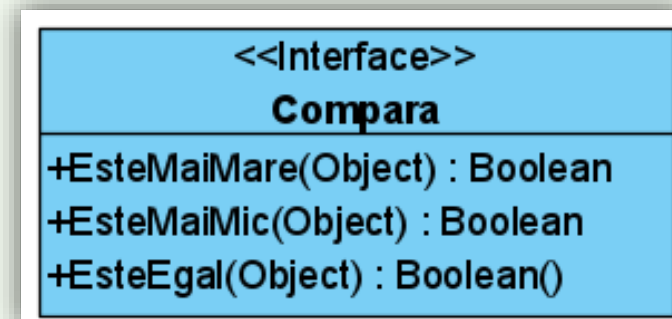
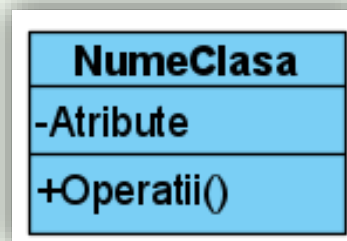
Caracteristicile unei clase sunt **atributele** și **operațiile**, prin care de descriu proprietăți și comportamente .



Clasele **abstracte** nu pot fi instanțiate. Rolul lor este de a permite altor clase să le moștenească, în vederea reutilizării caracteristicilor.



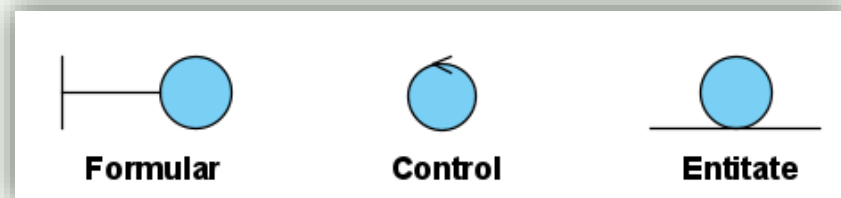
O **interfață** descrie un set de caracteristici și obligații publice. Specifică, de fapt, **un contract**. Orice instanță care implementează interfața trebuie să ofere serviciile furnizate prin contract.



Exemple de clase stereotipe uzuale

Clasele stereotipe conțin informații adiționale despre semnificația clasei, cu scopul de a face modelul mai ușor de înțeles.

- **entitate (<<entity>>)** – o clasă pasivă, care nu inițiază interacțiuni;
- **control (<<control>>)** – inițiază interacțiuni, conține o componentă tranzacțională și este separator între entități și limite;
- **limită (<<boundary>>)** – este aflată la periferia sistemului, dar în interiorul său. Reprezintă limita de legătură cu actorul sau cu alte sisteme informatice;
- **enumerare (<<enumeration>>)** - este folosită pentru definirea tipurilor de date ale căror valori sunt enumerate.
- **primitivă (<<primitive>>)** - o formă de clasă care reprezintă tipuri de date predefinite, cum ar fi tipul Boolean.



Atribute -1

- Fiecare atribut este descris cel puțin prin numele său.
- Se pot adăuga și informații adiționale, iar forma generală a unui atribut este:

[vizibilitate][/]nume[:tip][multiplicitate][=valoare implicită] [{proprietate}]

- Vizibilitatea poate fi:
 - + public: poate fi văzută și folosită de oricine
 - - private: numai clasa însăși poate avea acces
 - # protected: au acces clasa și subclasele acesteia
 - ~ package: numai clasele din același pachet pot avea acces
- / simbolizează un atribut derivat



Atribute -2

- UML permite specificare **multiplicităților pentru atribute**, atunci când dorim să definim mai mult de o valoare pentru un atribut. Au următoarea semnificație:

Multiplicitate	Sens
1	Exact 1 (implicit)
2	Exact 2
1..4	De la 1 la 4 (inclusiv)
3, 5	3 sau 5
1..*	Cel puțin unul sau mai mulți
*	Nelimitat (inclusive 0)
0..1	0 sau 1

- Proprietate indică o **proprietate suplimentară** care se aplică atributului:
 - {readonly}: atributul poate fi citit, dar nu modificat
 - {ordered}, {unordered}: o mulțime ordonată sau neordonată
 - {unique}, {nonunique}: mulțimea poate conține sau nu elemente identice

Operații

- Forma generală a unei operații este:

[vizibilitate] nume ([direcție] lista parametri) [:tip returnat] [{proprietate}]

- Vizibilitatea – aceeași ca și la clase
- Direcție - 'in' | 'out' | 'inout' | 'return'
- Tip returnat – dacă operația returnează ceva, adică este o funcție
- Un exemplu de proprietate a unei operații: {query} - nu are efecte secundare, nu schimbă starea unui obiect sau a altor obiecte, exemplu operațiile de tip “get”.

Exemple de attribute:

- - varsta: Integer {varsta>18}
- # nume:String[1..2]="Ioana"
- ~ Id:String {unique}
- / sumaTotala:Real=0

Exemple de operații:

- + setVarsta (out varsta: Integer)
- + getVarsta(in Id:String): Integer {query}
- - schimbaNume(inout nume:String)

Constrângeri

- O constrângere este o expresie care **restricționează** un anumit element al diagramei de clase.
- Aceasta poate fi o **expresie formală** (scrisă în Object Constraint Language - OCL) sau o formulare **semi-formală** sau **informală**.
- Acestea sunt reprezentate între acolade.
- Pot fi scrise imediat după definirea elementului sau ca un comentariu.
- O constrângere poate avea și un nume, astfel: **{nume : expresie booleană }**

Exemple de constrângeri OCL:

context Organizatie

inv: self. departamente→isUnique (nume)

inv: departamante.angajati→isUnique (cod)

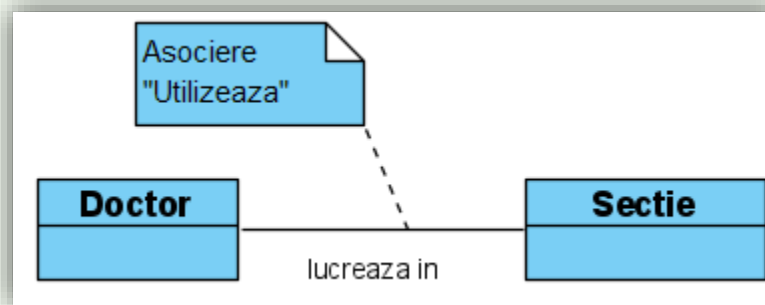
Produs

-nume : String {nuVid:nume->NotEmpty()}
-pret : Real {pret>0}

Relația de asociere -1

1. Asocierea implică stabilirea unei relații între clase.

- ✓ Relațiile de asociere între clase pot fi clasificate ca „utilizări” - deoarece reprezintă o clasă folosind o altă clasă într-un anume fel.
- ✓ Asocierea este mai cea **frecventă** și cea mai **comună** relație între două clase în orientarea obiect.
- ✓ Cele mai multe clase de asociere sunt **coezive** - adică lucrează împreună pentru a atinge un obiectiv în cadrul sistemului.

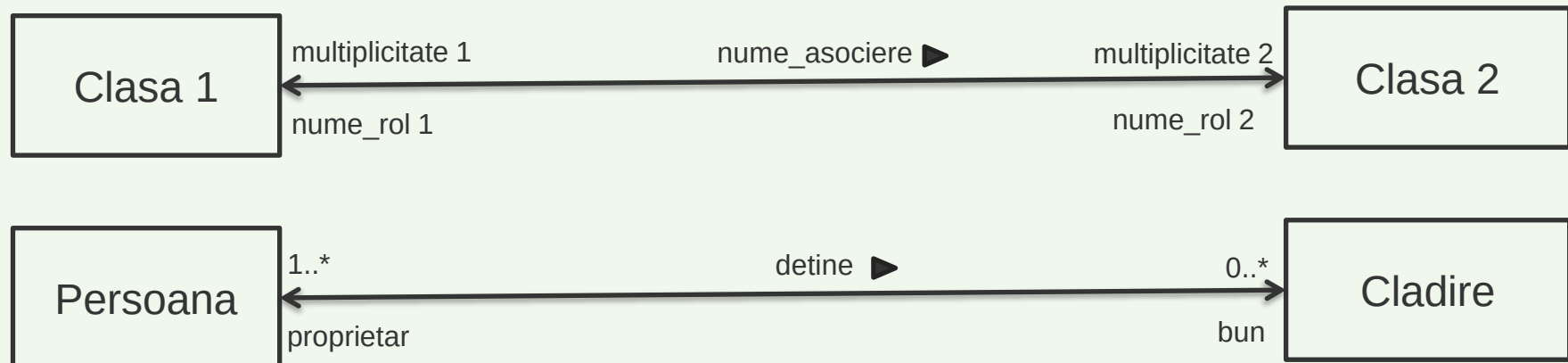


Doctor *utilizeaza* Sectie. Sectie *utilizeaza* Doctor. Este o relație bilaterală.



Relația de asociere -2

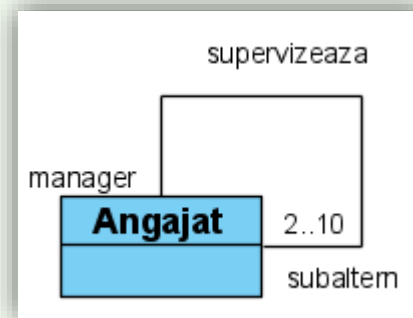
- Asocierea dintre clase este caracterizată prin:
 - ✓ **Denumire** (opțională): reflectă natura relației dintre clase.
 - ✓ **Multiplicități**: arată **câte obiecte** alei unei clase care pot fi asociate cu **un obiect** al clasei de la capătul opus al asocierii; se trec la cele două capete ale asocierii.
 - ✓ **Roluri ale asocierii**: se trec la fiecare capăt al asocierii și conțin o descriere scurtă și reprezentativă de 1 – 2 substantive; arată modul de implicare a obiectului în relație.
 - ✓ **Direcție de navigare**: legată de modul în care obiectele se pot accesa unele pe altele.



Tipuri de asocieri

- **Unare (reflexive)**

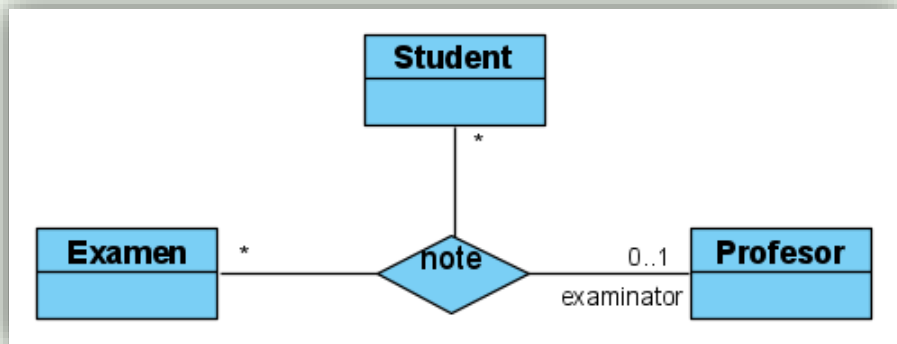
- ✓ Asocieria reflexivă este utilizată atunci când pot fi asociate obiecte din aceeași clasă.
- ✓ Notăția rămâne aceeași, cu excepția faptului că linia de asociere este trasă către și din aceeași clasă.



- **Binare** (între două clase)
- **N-are** (între trei sau mai multe clase) – apar atunci când mai mult de două obiecte sunt implicate într-o relație.

Asocieri n-are -1

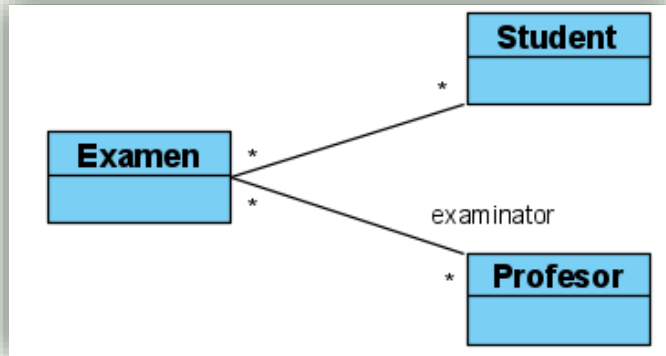
- O asociere n-ară este reprezentată ca un romb gol în centru. Rombul este conectat cu toate clasele participante prin intermediul unei asocieri nedirecționată.
- Nu există direcții de navigare pentru asocierile n-are; Se pot specifica multiplicități și nume de roluri.



Varianta 1: Modelarea unei asocieri ternare care arată relația dintre Student, Examen și Profesor.

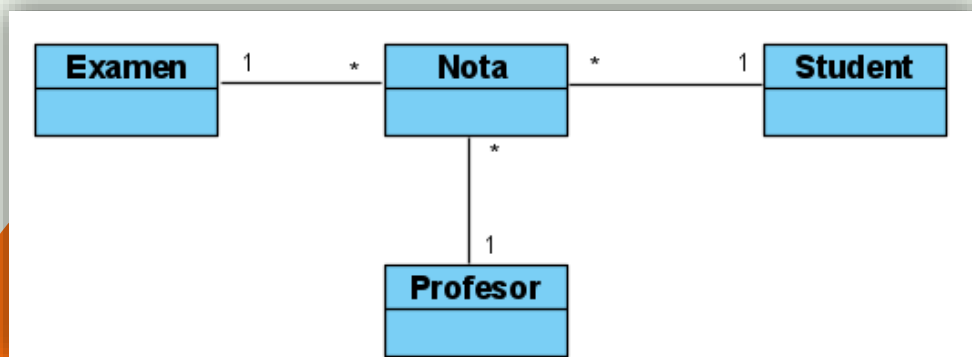
- ✓ Un **Student** susține un **Examen** fără **Profesor** (nu susține deloc acest examen) sau cu exact un Profesor.
- ✓ Un **Examen** cu un **Profesor** poate fi susținut de orice număr de **Studenți**.
- ✓ Un **Student** poate fi notat de un **Profesor** pentru orice număr de **Examene**.
- ✓ *Nu este posibil ca doi sau mai mulți **Profesori** să noteze un **Student** pentru același **Examen**.*

Asocieri n-are -2



Varianta 2: Transformarea asocierii ternare în două asocieri binare va conduce la un model cu o semnificație diferită.

- ✓ Un examen poate fi notat de mai mulți profesori.
- ✓ Nu se arată în mod clar ce Profesor a evaluat un anumit Student la Examen.
- ✓ Varianta 1: studentul s1 a susținut examenul e1 cu profesor p1 și că studentul s2 a susținut același examen e1 cu profesor p2.
- ✓ Varianta 2: se poate exprima doar că studenții s1 și s2 au susținut examenul e1 și că examenul e1 are doi examinatori p1 și p2. Cu acest model, nu puteți exprima ce profesor notează pe ce student.

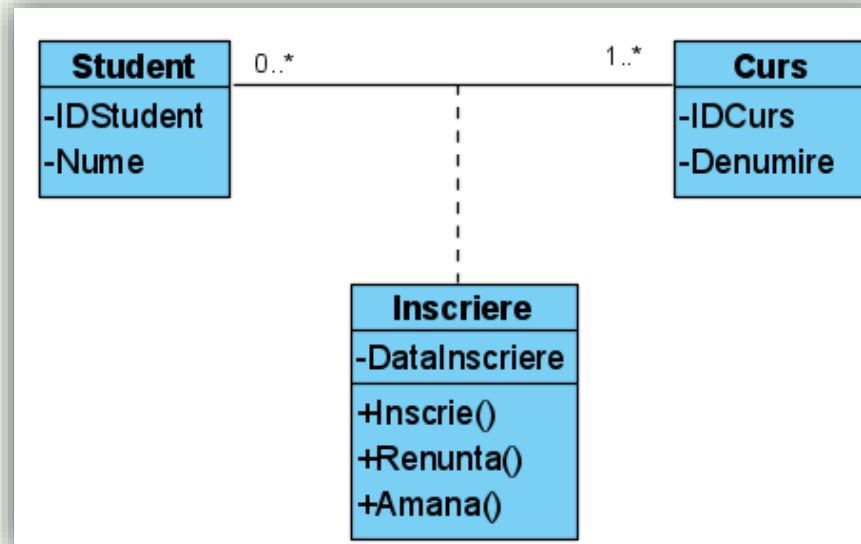


Varianta 3: Poate fi introdusă o clasă suplimentară pentru notare.

- ✓ În acest model este posibil ca un student să fie notat de mai multe ori pentru același examen ceea ce nu este posibil la modelul cu asociere ternară.

Clasă modelată ca asociere

- O clasă modelată ca o asociere este utilizată pentru a surprinde anumite **caracteristici ale unei asocieri** între două clase.
- Aceste caracteristici nu aparțin claselor asociate, ci **aparțin relației dintre clase**.
- Clasele modelate ca asocieri au sens numai dacă reprezintă asocieri cu un anumit **comportament și stare**.
- Fiecărei instanțe a unei clase modelată ca asociere ar trebui să îi corespundă o **pereche unică** de obiecte ale claselor care participă la asociere.
- Necesară atunci când modelăm **asocieri mulți-la-mulți**.



Relația de agregare

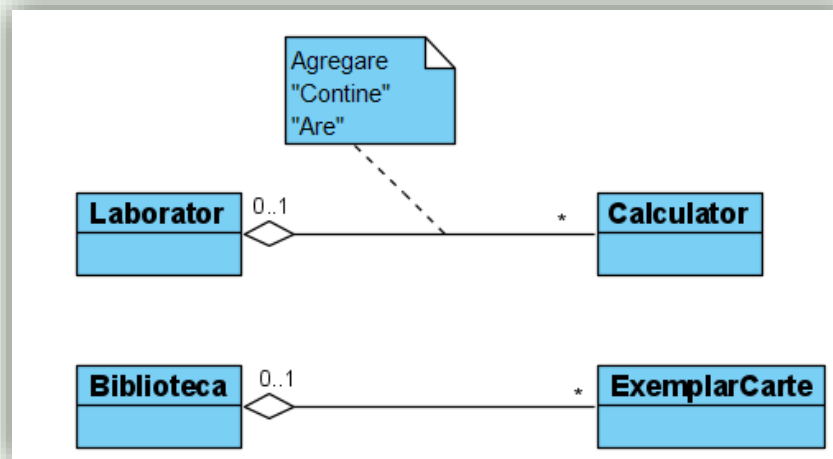
2. Agregarea este o formă de asociere binară reprezentând o relație de tip **parte/întreg**.

- ✓ Relația de asociere indică o relație relativ slabă între două clase. Această relație “liberă” înseamnă că obiectele aparținând acestor clase asociate pot exista **independent** unul de celălalt.
- ✓ Dacă relația dintre două clase asociate este strânsă, atunci aceasta este reprezentată de agregare. Agregarea este astfel o **formă specială de asociere**.
- ✓ Agregarea reprezintă o clasă care **conține** o altă clasă
- ✓ Are semnificații precum: “**Este format din**”, “**Conține**”, “**Are**”.
- ✓ Există două forme ale relației de agregare:
 - Agregare **partajată**
 - Agregare **compusă**



Agregarea partajată

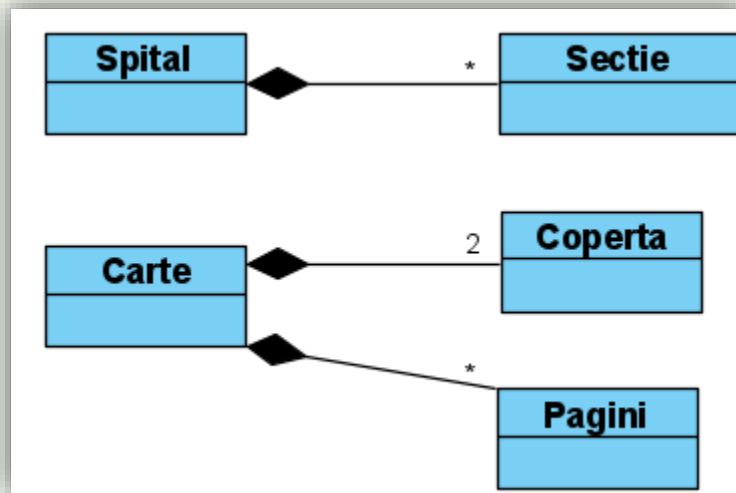
- Agregarea partajată este o **formă slabă** de agregare în care instanțele părților sunt independente de întreg, astfel:
 - ✓ Aceleași părți partajate pot fi incluse în **mai multe** clase întreg.
 - ✓ Dacă clasa întreg se șterge, clasele parte **vor exista** în continuare.
 - ✓ Se reprezintă sub forma unui **romb gol** plasat la capătul clasei întreg.



Laborator *conține* Calculator.
Biblioteca *are* Exemplar Carte
Este o relație mai strânsă.

Agregarea compusă

- Agregarea compusă este o formă **puternică** de agregare în care instanțele părților sunt dependente de întreg, astfel:
 - ✓ Dacă clasa întreg se șterge, clasele parte **vor fi șterse** și ele.
 - ✓ Se reprezintă sub forma unui **romb plin** plasat la capătul clasei întreg.
 - ✓ Atunci când se folosește pentru modelarea obiectelor dintr-un anumit domeniu, ștergerea poate fi **interpretată la figurativ**, ca “terminare”, și nu ca o distrugere fizică.



Legătura între asociere, agregare și compunere

Asociere

Obiectele știu unele de existența celorlalte și pot lucra împreună.

Agregare

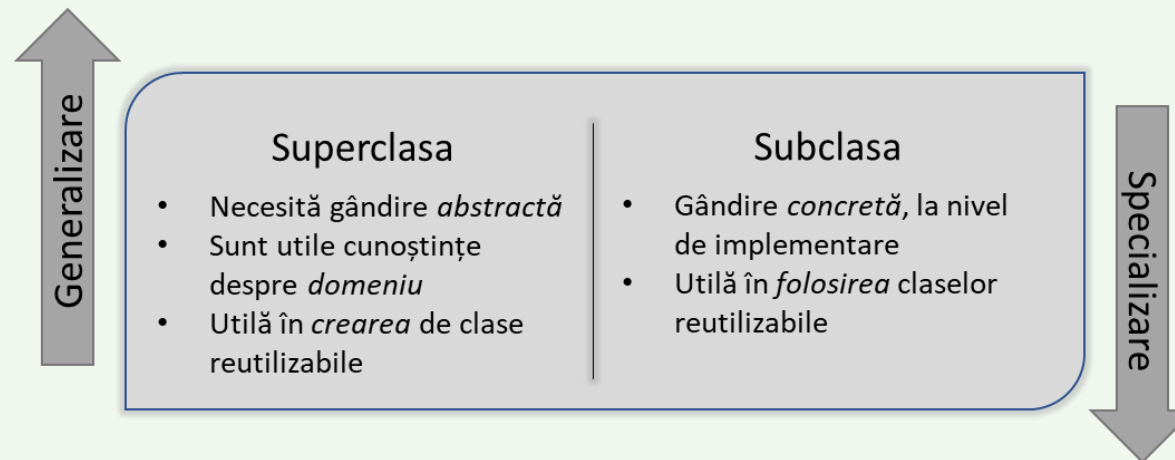
1. Protejează integritatea configurației.
2. Funcționează ca un tot unitar.
3. Control prin intermediul unui singur obiect.

Compunere

Fiecare parte poate fi membră a unui singur obiect agregat.

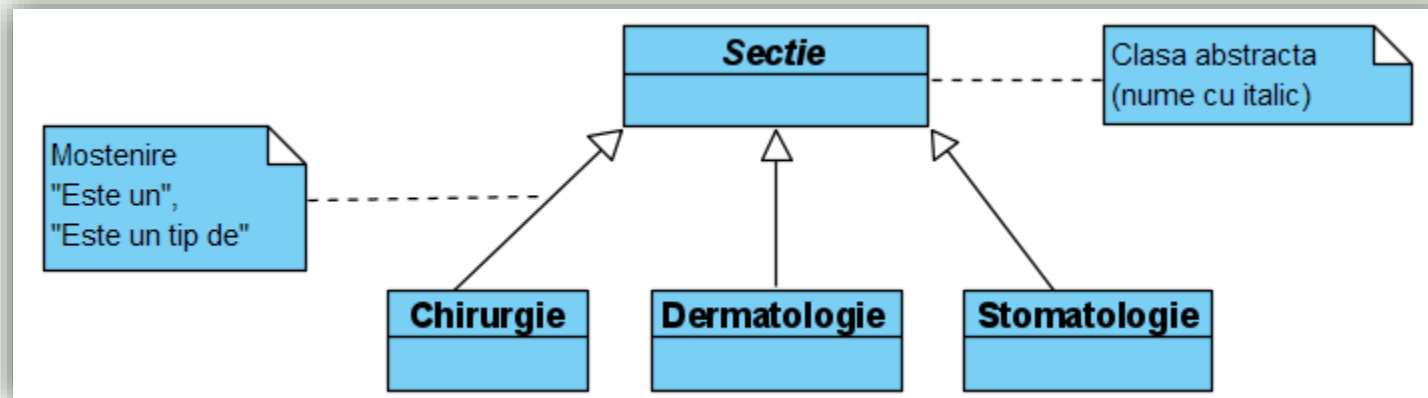
Relația de generalizare -1

- Căutarea **elementelor comune** în cadrul atributelor și operațiilor unui grup de clase și plasarea acestora într-o clasă comună sau generică se numește **generalizare**.
- Deplasarea în sus a ierarhiei moștenirii necesită **cunoașterea domeniului** și gândirea **abstractă**.
- Când clasele sunt **derivate** pe baza claselor existente în clase specializate, se numește **specializare**.
- Specializarea necesită o gândire **concretă** - deplasarea spre implementare.



Relația de generalizare -2

- Aceste clase de nivel **superior-inferior** sunt, de asemenea, cunoscute sub numele de super-sub, părinte-copil sau clase derivate- de bază. Frecvent, super-clasele sunt clase **abstracte**.
- Se mai numește informal și relație de genul “**este un tip de**” sau “**este un**”.
- Se reprezintă sub forma unui **triunghi gol** plasat la capătul superclasei.
- Este permisă **moștenirea multiplă**.



Sugestii pentru denumirea claselor

- ✓ Numele clasei este reprezentat de un **substantiv la singular**.
- ✓ Numele claselor încep cu **literă mare**.
- ✓ Numele claselor **abstracte** este scris cu **italic**.
- ✓ Pot exista și clase al căror nume este **compus** din două sau mai multe cuvinte (exemplu: PacientPrivat).
- ✓ **Stereotipul** unei clase apare ori sub formă de pictogramă specifică, ori având denumirea scrisă deasupra numelui clasei între simbolurile << >>.



Bune practici – Multiplicități

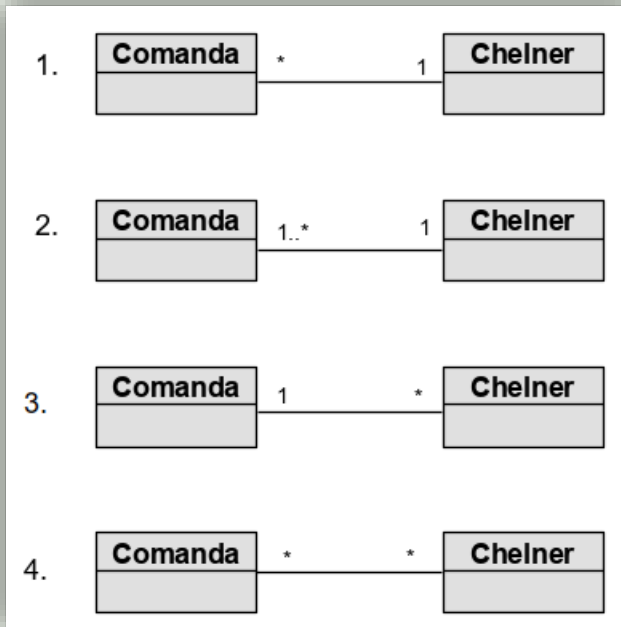
- ✓ Multiplicitățile indică **numărul de obiecte** ale unei clase legate de **un obiect** al altei clase.
- ✓ Multiplicitățile **au sens** atunci când două clase sunt în **asociere sau agregare**.
- ✓ Multiplicitățile indică, de asemenea, **reguli ale afacerii**.
- ✓ Rețineți că **NU** pot exista multiplicități între două clase care au o relație de **moștenire**.
- ✓ În cazul compunerii, o multiplicitate în clasa de nivel superior (clasa întreg), deși nu este greșită din punct de vedere sintactic, are o **semnificație semantică limitată**. Multiplicitatea corespunzătoare clasei întregi este **unu**.



Aprofundăm- 1

Cum modelați următoarea situație într-o diagramă de clase UML:

O comandă este plasată cu ajutorul unui chelner, un chelner se ocupă de mai multe comenzi.



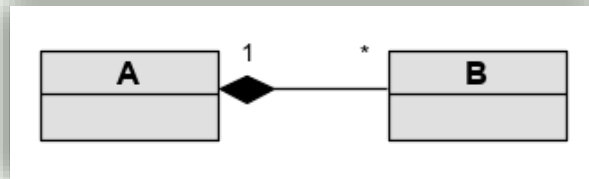
Variante:

- a) 1+2
- b) 1+3
- c) 1+4
- d) 2+3



Aprofundăm- 2

Pornind de la diagrama din imagine, care dintre următoarele afirmații este adevărată?



1. Un obiect al clasei A poate avea asociat un obiect al clasei B
2. Dacă o instanță a lui B este ștearsă, atunci sunt șterse și toate instanțele lui A
3. Rombul de lângă A poartă numele de compunere
4. Un obiect al lui B este inclus într-un sigur un obiect al lui A

Variante:

- a) 1+4
- b) 1+3
- c) 2+3
- d) 3+4



Aprofundăm- 3

O relație de generalizare între o subclasă și o superclasă are următoarele proprietăți:

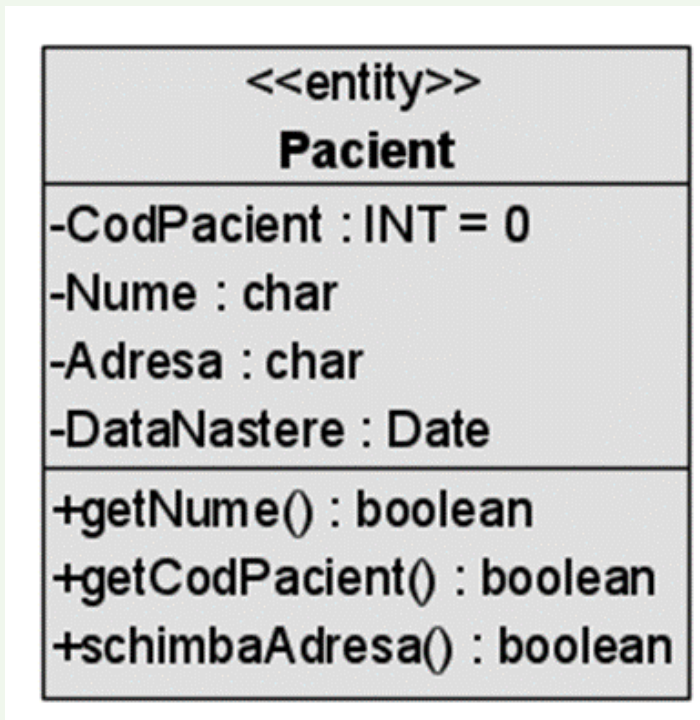
1. subclasa moștenește proprietățile superclasei
2. subclasă poate moșteni de la o singură superclasă
3. subclasa nu poate avea mai multe atribute și operații decât superclasa
4. superclasa nu poate să fie abstractă

Variante:

- a) 1
- b) 1+2
- c) 1+2+3
- d) 1+2+3+4



Aprofundăm- 4



În această reprezentare a unei clase identificați:

- Numele clasei
- Stereotipul
- Atributele
- Valoarea inițială
- Vizibilitatea atributelor
- Tipul atributelor
- Operațiile
- Semnătura operațiilor
- Vizibilitatea operațiilor
- Valoarea returnată

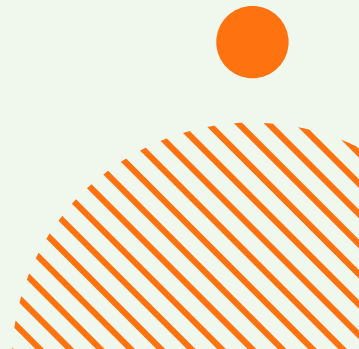


Studiu de caz

Scopul proiectului este realizarea unui sistem informatic pentru gestiunea activităților de rezervare și cazare ale unei unități hoteliere. În vederea cazării, un client poate solicita rezervarea uneia sau mai multor camere prin e-mail, telefonic sau folosind site-uri specializate în rezervări hoteliere. Pentru rezervările prin email sau telefonic, clientul furnizează recepționarului informații privind perioada de cazare și tipurile de camere solicitate. Clienții vor beneficia de reduceri implicite dacă rezervă cel un număr minim de camere sau dacă perioada de cazare depășește patru zile. Recepționarul verifică disponibilitatea camerelor și îl înștiințează pe client de acest lucru precum și de costul estimat al cazării. Dacă nu există camere disponibile conform solicitării, recepționarul poate oferi clientului alternative. De asemenea, clientul poate solicita un discount suplimentar, iar recepționarul va decide fezabilitatea discountului, fiind asistat obligatoriu de managerul hotelului. În situația în care clientul este de acord cu prețul propus, se va proceda la realizarea rezervării. Pentru clienții noi, recepționarul solicită datele de identificare, pe care le introduce în aplicație. Nu se poate acorda discount suplimentar pentru rezervările făcute prin intermediul site-urilor specializate.

Odată ajuns la hotel, și dacă a făcut în prealabil o rezervare, clientul va furniza datele de identificare ale sale și/sau ale rezervării și se face cazarea. Dacă nu există o rezervare, se va verifica disponibilitatea camerelor pentru perioada cerută. Atunci când se găsește o astfel de cameră, se face cazarea. La finalul sejurului, recepționarul întocmește o listă cu toate serviciile solicitate de client, precum și prețul acestora. Lista trebuie validată de client, după care se întocmește factura finală. Factura poate fi plătită parțial sau integral, prin transfer bancar, numerar sau folosind un card bancar. Totodată, înainte de a părăsi hotelul, clientul este rugat să completeze un formular prin care să evalueze serviciile oferite de unitatea hotelieră. După eliberarea camerei, camerista verifică starea acesteia și informează recepția.

➤ ***Să se reprezinte în Visual Paradigm diagrama de clase pentru scenariul de mai sus.***



De lucru

Realizați diagrame de clase pentru situațiile de mai jos:

1. O echipă este formată din 11 jucători, dintre care unul este căpitan.
2. O companie este formată din departamente. Departamentele sunt localizate în una sau mai multe clădiri de birouri. Unul dintre birouri funcționează ca sediu central. Fiecare departament are un manager care este recrutat din rândul angajaților săi.
3. Un student poate participa la mai multe cursuri susținute de un profesor. De asemenea, profesorul poate fi îndrumător studentului pentru lucrarea de disertație.
4. Un client poate obține un credit de la bancă numai dacă oferă o garanție imobiliară în acest sens. Creditul este considerat un serviciu oferit de bancă.
5. Un colecționar deține una sau mai multe colecții. Pentru a se constitui o colecție sunt necesare cel puțin 2 obiecte.
6. Într-o clinică medicală, un pacient care prezintă anumite simptome, poate realiza programări la medicii clinicii. Atât medicii cât și pacienții trebuie să fie utilizatori ai sistemului.